

Smart Contract Audit Report

MoonsalePresale + Factory

Solidity ^0.8.20 · EIP-1167 proxy pattern

REPORT ID	IGH-MSL-2026-013
CLIENT	Moonsale App
SCOPE	Presale + Factory + Interface (see Scope section)
VERSION	Round 3 submission (post IGH-MSL-2026-006)
METHODOLOGY	Manual review, property verification, regression scan
ISSUE DATE	April 23, 2026

Executive Summary

This is a comprehensive Round 3 review of the MoonSale presale stack, covering `MoonsalePresale.sol` (the EIP-1167 implementation), `MoonsaleFactory.sol` (factory plus LP locker), and the shared `IMoonsalePresale.sol` interface. The submission follows the remediations already tracked in IGH-MSL-2026-003 (initial factory audit) and IGH-MSL-2026-006 (factory follow-up), and the client has supplied a detailed submission brief enumerating twelve security properties to verify and three acknowledged open items pending before mainnet.

All twelve properties verify against the source. The emergency-refund grace period, dual-condition refund logic, DEX pre-seeding protection, creation-time slippage guards, fee-on-transfer rejection, excess-ETH refund, pull-based listing fee, leftover-liquidity returns, vesting math, early-withdrawal penalty cap, cancel in Filled state, and manual reentrancy guard are all implemented correctly and behave as specified. The manual reentrancy guard in the presale uses the pattern required by EIP-1167 clones (storage-level `_locked` flag rather than `ReentrancyGuard`'s constructor-initialized state), and all ETH-sending functions are consistently gated. The factory continues to be in the state reviewed under IGH-MSL-2026-007 with the three items the client has explicitly acknowledged as open.

No Critical, High, or Medium findings are introduced by this round. One new Low severity finding is raised: the per-presale `platformFeeRecipient` push transfer inside `finalize()` creates a narrow denial-of-service vector if the recipient address (baked into each clone at creation) becomes reverting between creation and finalization. Six Informational items cover a dead event declaration, unclaimed router approval residue, a participant count that never decrements, and several minor cosmetic points. None of these block mainnet deployment.

Findings Overview (New This Round)

Severity	Count	Notes
Critical	0	—
High	0	—
Medium	0	1 acknowledged open (M-01 from IGH-MSL-2026-006)
Low	1	Plus 2 acknowledged open (L-01, L-03 from prior round)
Informational	6	Plus 5 acknowledged open (I-01 through I-05 from prior round)

Overall Security Score

91 / 100

factory was 92; presale was 87

Scope

The following files are in scope for this round. All three must be reviewed together: the presale imports the interface, the factory imports the presale.

File	Lines	Role
MoonsalePresale.sol	~497	EIP-1167 presale implementation
MoonsaleFactory.sol	~377	Factory plus LP locker (reaffirmed from IGH-MSL-2026-007)
IMoonsalePresale.sol	43	Events, enums, function signatures

On-Chain Verification

The client provided verified source addresses on public testnets. Contract deployment matches the submitted source at both verification points. Mainnet deployment addresses will require fresh verification once published.

Network	Role	Address
Sepolia	Presale Implementation	0x2D20Fc2e0cafC58B499ABEa0d9036BF99330E8c8
Sepolia	Factory	0x950Ea01Ab0d283f6d400CEB29c5d43b92725E0D4
BSC Testnet	Presale Implementation	0x2f098c345B297173b7C6AFF2038102AEB0ae2b98
BSC Testnet	Factory	0xFB6c08086B017117b9ba0fb8b5Ea1997e3A8aD34

Compiler Configuration

Setting	Value
Solidity	^0.8.20
EVM Version	paris
Optimizer	enabled, 200 runs

Property Verification

The submission brief specifies twelve security properties. Each was independently verified against the source; all twelve hold.

#	Property	Status	Key Evidence
1	Emergency refund 30-day grace period	VERIFIED	emergencyRefund() gates on endTime + FINALIZE_GRACE_PERIOD
2	Dual _isRefundable() condition (no gap)	VERIFIED	$A \cap B = \emptyset$ (softcap split); no status falls between
3	DEX pair pre-seeding protection	VERIFIED	getReserves() == 0 required; revert rolls back status
4	Creation slippage guards (M-01 v1)	VERIFIED	Both create functions check fee + router before state write
5	Fee-on-transfer token rejection	VERIFIED	balBefore on presale addr; strict equality required
6	Excess ETH refund on creation	VERIFIED	msg.value - listingFee refunded; nonReentrant-protected
7	Pull-based listing fee (M-02 v1)	VERIFIED	accumulator invariant holds; sweepStuckETH subtracts
8	Leftover DEX tokens/ETH to creator (M-04)	VERIFIED	creatorProceeds = afterFee - usedETH; leftovers returned
9	Vesting math (no over-claim)	VERIFIED	Floor arithmetic; sum of allocations <= total
10	Early withdrawal penalty cap	VERIFIED	25% cap enforced locally; Filled state blocks withdraw
11	Cancel in Filled state	VERIFIED	cancelPresale allows Pending/Active/Filled
12	Manual reentrancy guard	VERIFIED	_locked flag; all ETH paths guarded

Notable Detail: Property 3 (Pre-Seeding Protection)

The reserve check in `finalize()` executes after the line `status = Status.Finalized`. A reader might worry this leaves the presale stuck in Finalized state when the check fails. It does not: a `require` in Solidity rolls back the entire transaction, including the earlier status assignment. The presale returns to its pre-call state (Active, Filled, or Pending) and `cancelPresale()` remains callable. This is the correct behaviour the brief specifies.

Notable Detail: Property 9 (No Over-Claim)

The key arithmetic property is that per-investor claim allocations sum to no more than the `tokensForDistribution` reserved at finalization. Each per-investor total is `floor(contribution * presaleRate / 1e18)`, and by the standard floor-sum inequality $\sum \text{floor}(x_i * r / 1e18) \leq \text{floor}(\sum x_i * r / 1e18)$, the sum of allocations is bounded above by `tokensForDistribution`. The inequality may be strict when division rounds down, producing small dust savings retained by the contract; over-claim is impossible. The `tokensClaimed` per-investor counter prevents double claims within the vesting curve.

Notable Detail: Property 12 (Manual Reentrancy Guard)

OpenZeppelin's `ReentrancyGuard` initializes state in its constructor, which EIP-1167 minimal proxies do not execute. Using it with clones would leave `_status` uninitialized, causing either permanent locking or no protection at all depending on the default value semantics. The manual `bool private _locked` pattern with the modifier setting and clearing the flag is the correct approach for clone-based contracts and is applied consistently to all ETH-sending functions in the presale.

Remediation Status — Prior Audits

Findings from both prior audits are re-verified. Status is based on code inspection, not self-report. Three items remain open and are acknowledged by the client as pending fixes scheduled before mainnet (per the submission brief). These three items are listed but their full descriptions are not repeated here.

From IGH-MSL-2026-003 (initial factory audit, 72/100)

Prior ID	Finding (short)	Status
H-01	No emergency refund for ghosted creator	RESOLVED
H-02	DEX pair pre-seeding blocks finalization	RESOLVED
M-01	Excess ETH stuck in factory on creation	RESOLVED
M-02	Push listing fee DoS	RESOLVED
M-03	No slippage guard on creation	RESOLVED
M-04	Leftover DEX tokens/ETH stuck	RESOLVED
L-01	No upper bound caps on admin params	ACCEPTED
L-02	getAllPresales unbounded array	ACCEPTED
L-03	No emergency pause on creation	RESOLVED
L-04	No extendLock for LP owners	RESOLVED
L-06	Fee-on-transfer silent pool drain	RESOLVED

From IGH-MSL-2026-006 (factory follow-up, 89/100)

Prior ID	Finding (short)	Status
M-01	setFeeRecipient doesn't drain accumulator	OPEN (ACK)
L-01	No expectedFeeRecipient guard on create	OPEN (ACK)
L-02	extendLock and sweepStuckETH lack nonReentrant	RESOLVED
L-03	extendLock lacks max unlock time cap	OPEN (ACK)
I-01	Constructor doesn't emit initial-config events	OPEN (ACK)
I-02	Duplicated logic in create functions	OPEN (ACK)
I-03	Single-key Ownable	OPEN (ACK)
I-04	Non-deterministic clone addresses	OPEN (ACK)
I-05	sweepStuckETH destination coupled to feeRecipient	OPEN (ACK)

Status legend. **Resolved** = fix verified in source. **Accepted** = client has permanently accepted the risk with compensating control. **Open (ack)** = client acknowledges the finding and has stated it will be fixed before mainnet; auditor has not been asked to re-describe. Full descriptions remain in the referenced prior report.

New Findings (This Round)

The items below are new findings raised during this round's review. None rise to Critical, High, or Medium severity. They are independent of the three acknowledged open items from IGH-MSL-2026-006.

Low

LOW

L-01 platformFeeRecipient push transfer in finalize() creates per-presale DoS vector

Description.

`finalize()` pays the platform fee via a direct push: `platformFeeRecipient.call{value: fee}("")` on line 218, with `require(feeOk)` on line 219. Each presale clone stores the fee recipient address at initialization time (line 129 of the presale). If this address becomes non-receiving between creation and finalization (for example, a multisig reconfigured to reject native-token sends, or a contract upgraded to revert on fallback), all affected presales can no longer finalize. Contributors are not permanently harmed, since the creator can call `cancelPresale()` and contributors can then `refund()`, or after 30 days anyone can call `emergencyRefund()`. However, the intended successful outcome (liquidity added, tokens distributed) is blocked. The pattern is asymmetric with the pull-based model used for the factory's listing fees (resolved as M-02 in IGH-MSL-2026-003).

Recommendation.

Consider a pull-based fee pattern for the per-presale platform fee as well. Add a `pendingPlatformFee` storage field, credit it in `finalize()` instead of pushing, and expose a `claimPlatformFee()` callable by `platformFeeRecipient`. Alternatively, if the push model is retained, document the operational requirement that `platformFeeRecipient` must remain reliably ETH-receiving for the lifetime of all active presales it has been baked into. Either approach is acceptable; the pull model is more robust.

Informational

INFO

I-01 Creator token over-deposit becomes stranded after finalization

Description.

`finalize()` requires `token.balanceOf(this) >= tokensForLiquidity + tokensForDistribution` (line 228), permitting over-deposit. Post-finalization, only `leftoverTokens = tokensForLiquidity - usedToken` (router slippage residue) is returned to the creator. Any over-deposit above the required sum remains in the presale contract indefinitely: `withdrawCreatorTokens()` only operates when status is Failed, Cancelled, or refundable, not Finalized. This is a self-inflicted UX issue for the creator, not a vulnerability in the protocol's sense.

Recommendation.

Two acceptable paths. First, add a creator-only recovery function callable after finalization that computes `strandedTokens = balanceOf(this) - sum(tokensClaimable across investors)` and allows the creator to withdraw the difference. The sum can be tracked incrementally rather than computed on demand. Second, document the "exact-deposit-required" expectation prominently in integrator documentation and surface it in the front-end.

INFO

I-02 Clone address is predictable, enabling a token-prefund grief vector**Description.**

`Clones.clone` (non-deterministic) derives the new address from the factory's nonce, which is publicly readable. A mempool observer could predict the next clone address after seeing a `createPresaleAndDeposit` transaction, front-run with a token transfer to that predicted address, and cause `balBefore > 0`. The factory's FoT detection check `received == depositAmount` still passes (the math works out), but the attacker-transferred tokens become permanently stranded in the presale. This is pure grieving: the attacker loses their tokens and the presale's effective token pool is inflated by a value the creator did not intend. Not exploitable for profit.

Recommendation.

Optional. If the team wishes to harden against this, switch to `Clones.cloneDeterministic` with a commit-reveal pattern where the creator supplies a salt, or reject creations where `balBefore > 0`. Both add friction for legitimate users. Given the attacker must sacrifice tokens with no profit path, leaving this as is is defensible.

INFO

I-03 Router approval residue after `addLiquidityETH` is not cleared**Description.**

`finalize()` approves the DEX router for `tokensForLiquidity` at line 242, and the router consumes `usedToken <= tokensForLiquidity`. Residual approval for `tokensForLiquidity - usedToken` remains. Because line 270-273 transfers the leftover tokens to the creator, the presale no longer holds tokens the router could pull. However, if the presale ever receives additional tokens of the same type through an unexpected path (direct transfer, for example), the residual approval would allow the router to pull them. This is a defense-in-depth concern rather than an active vulnerability.

Recommendation.

Add `token.forceApprove(dexRouter, 0)` after the `addLiquidityETH` call to zero out the residue. Gas cost is negligible and the cleanup aligns with best practice.

INFO

I-04 `EmergencyWithdraw` event declared in interface but never emitted**Description.**

The interface `IMoonsalePresale.sol` declares event `EmergencyWithdraw(address indexed to, uint256 amount);` at line 24. No implementation in the presale contract emits this event. Downstream consumers (indexers, monitoring tools, front-ends) may subscribe to it and receive no data, wasting indexer resources and creating a confusing developer experience.

Recommendation.

Remove the declaration from the interface if no emergency-withdraw flow is planned, or add the corresponding emit in the function where it applies. `emergencyRefund()` already emits `EmergencyCancelled` and `Refunded`, so this declaration appears to be a vestige of an earlier design.

INFO

I-05 participantCount never decrements on refund or withdrawal**Description.**

participantCount increments in `contribute()` when a new address first contributes. It is never decremented in `refund()`, `withdrawContribution()`, or `emergencyRefund()`. The view therefore reports the cumulative count of unique participants ever, not the count of currently-holding participants. This is a cosmetic issue and may even be the intended interpretation for analytics purposes, but the name participantCount is ambiguous.

Recommendation.

Either rename the variable to `uniqueContributorsCount` to match current behaviour, or add decrement logic on the three withdrawal paths with `_isParticipant[msg.sender] = false` when `contributions[msg.sender] == 0`. The first option is less invasive.

INFO

I-06 isPresaleActive() returns false in Filled state**Description.**

`isPresaleActive()` returns true only when status is Pending or Active. When the hardcap is reached and status transitions to Filled, the view returns false even though the sale has not yet been finalized or cancelled. Front-ends using this as a "is the sale live" flag will show a presale as inactive while contributors could still potentially see their tokens distributed via `finalize()`. The semantic meaning is clear to contract readers (active = can still accept contributions), but the name invites misinterpretation.

Recommendation.

Add a complementary `isAcceptingContributions()` view with the current semantics, and change `isPresaleActive()` to include the Filled state so front-ends treat "filled but not finalized" as still live. Alternatively, document the name's intent in a NatSpec comment.

Positive Observations

- Twelve security properties explicitly specified by the client were all verified to hold in the source. The submission brief is exceptionally thorough and made the review efficient.
- Manual reentrancy guard correctly adopts the clone-compatible pattern rather than trying to use OpenZeppelin's ReentrancyGuard (which would fail with EIP-1167 proxies due to constructor skip).
- Dual `_isRefundable()` condition provides a clean, non-overlapping refund surface that covers both softcap-missed and creator-ghosted-after-softcap-met scenarios without a coverage gap.
- Emergency refund's 30-day grace period is a thoughtful investor-protection mechanism that prevents indefinite fund lockup if the creator ghosts.
- Cancel in Filled state is a deliberate design choice that gives creators an exit hatch after hardcap but before finalization. Investors retain full refund rights.
- Leftover-token and leftover-ETH from `addLiquidityETH` are both correctly returned to the creator, avoiding the common mistake of using `nativeForLiquidity` instead of the actual `usedETH` in the proceeds calculation.
- DEX pair pre-seeding protection is implemented with a strict zero-reserves check, catching even 1-wei pre-seeds that would skew the initial price.
- Status machine is clean and has no transitions that leave funds unreachable. Every path from a non-terminal status reaches either Finalized (claim), Failed (refund), or Cancelled (refund).
- On-chain verified source on both Sepolia and BSC Testnet matches the submitted files, confirming the audited artifact is the deployable artifact.
- Platform fee percent and penalty percent are capped at compile time via `MAX_FEE` and `MAX_PENALTY` constants and re-validated on every admin setter call.
- Initialization validates all sixteen parameters comprehensively, rejecting malformed presale configurations at creation time rather than at finalization.
- The early-withdrawal penalty is capped at 25% locally in the presale regardless of what the factory sets, providing a belt-and-suspenders protection against factory misconfiguration or upgrade.

Out of Scope

Per the submission brief, the following are out of scope for this round:

- Frontend code and Supabase database.
- `MoonsaleFairLaunch.sol`, `MoonsaleFairLaunchFactory.sol` (separate submission).
- `MoonsaleTokenLock.sol` (separately reviewed in IGH-MSL-2026-004/008/009).
- `MoonsaleTokenVesting.sol` (separately reviewed in IGH-MSL-2026-005/011/012).
- `MoonsaleTokenFactory.sol`, `MoonsaleToken.sol`, `MoonsaleLiquidityToken.sol`.

Recommended Remediation Order

#	Finding	Effort	Priority
1	Prior M-01: drain accumulator in <code>setFeeRecipient</code>	Low	High (pre-mainnet)
2	Prior L-01: add <code>expectedFeeRecipient</code> guard	Low	High (pre-mainnet)

#	Finding	Effort	Priority
3	Prior L-03: extendLock max unlock time cap	Low	Medium (pre-mainnet)
4	This L-01: pull-based platformFeeRecipient in finalize	Medium	Medium
5	This I-03: clear router approval post-addLiquidity	Low	Low
6	This I-04: remove dead EmergencyWithdraw event	Low	Low
7	This I-01, I-05, I-06: documentation + UX polish	Low	Low
8	This I-02: defer or accept clone-prefund grieving	—	Optional

Summary

The MoonSale presale stack has matured over three audit rounds into a well-structured, consistently-guarded contract system. The comprehensive submission brief specified twelve security properties to verify; all twelve hold against the provided source. The client has been transparent about three open items they plan to fix before mainnet, and those items are acknowledged without being re-described. No Critical, High, or Medium findings are introduced by this round.

One new Low severity finding raises the question of whether the per-presale platform fee transfer in `finalize()` should adopt the same pull-based pattern that the factory's listing fee uses. Six Informational items cover hygiene, cosmetic, and defense-in-depth improvements. None block deployment.

Deployment posture: ready for mainnet after the three client-acknowledged items (M-01, L-01, L-03 from IGH-MSL-2026-006) are fixed. The new findings in this report can be addressed in a follow-up polish release. User funds are protected against every vector identified across all three audit rounds.

Security Score	91 / 100	0 Critical · 0 High · 0 Medium · 1 new Low · 6 new Info
----------------	----------	---

Disclaimer

This report reflects a point-in-time Round 3 review of the Solidity source code provided by the client on the issue date shown on the cover. It covers only the files listed under Scope; contracts referenced but not provided are treated under the assumptions in Out of Scope. A successful audit does not guarantee the absence of vulnerabilities, and changes to the code after review invalidate the conclusions. This report is an advisory document and does not constitute financial, investment, legal, or tax advice. ICOGemHunters accepts no liability for losses arising from the non-implementation of recommendations contained herein, from implementation errors introduced when applying recommendations, or from interactions with contracts outside the reviewed scope. The client is solely responsible for deployment, operation, and ongoing security maintenance of the contract.

The MoonSale presale stack has completed three audit rounds across the presale and factory. Combined with the completed reviews of MoonsaleTokenLock (IGH-MSL-2026-009) and MoonsaleTokenVesting (IGH-MSL-2026-012), the core Moonsale platform is materially de-risked. When the three acknowledged open items are fixed, a short targeted follow-up can be requested for final sign-off.